



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 4

Title:

Explore the Docker architecture, Docker commands and Dockerfile and containerize a simple Python Application in Docker.

Aim:

To package a Python Flask application and its dependencies into a Docker image, run the application inside a Docker container, verify port mapping and application health, inspect logs and container contents, and demonstrate the Docker container lifecycle.

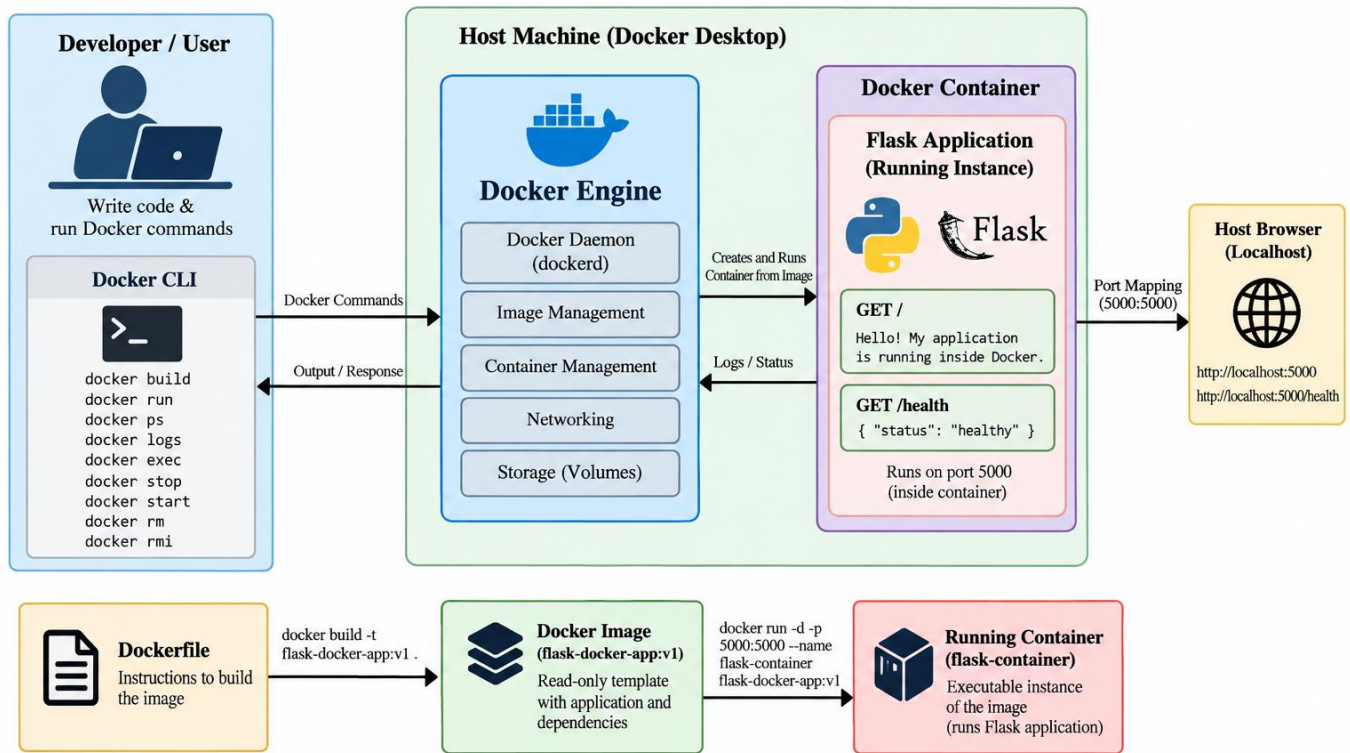
Theory:

Docker is a containerization platform used to package an application together with its runtime dependencies into a portable image. A Docker image is a read-only template used to create containers, while a container is a running instance of an image.

A Dockerfile contains the instructions used to build the image. Port mapping allows an application running inside a container to be accessed from the host machine.

For this assignment, a Python Flask application was tested locally first and then migrated into Docker. The application exposes port 5000 and provides two endpoints: / for the main response and /health for a health check. The .dockerignore file prevents unnecessary files from being included in the Docker build context.

Docker Architecture:



Code / Implementation:

Project Structure

Assignment-4/

- ├─ app.py
- ├─ requirements.txt
- ├─ Dockerfile
- ├─ .dockerignore
- └─ README.md

Flask Application — app.py

```
from flask import Flask  
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Hello! My application is running inside Docker."
```

```
@app.route("/health")
def health():
    return {"status": "healthy"}
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dependency File — requirements.txt

Flask

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

.dockerignore

```
.git
.gitignore
__pycache__
*.pyc
*.log
.env
venv
.vscode
README.md
```

Implementation Steps

Step	Activity	Command / Evidence
1	Docker verification	docker --version, docker info, docker run hello-world
2	Local application test	pip install -r requirements.txt; python app.py; verify / and /health
3	Build image	docker build -t flask-docker-app:v1 .
4	Verify image	docker images
5	Run container	docker run -d -p 5000:5000 --name flask-container flask-docker-app:v1
6	Check container	docker ps
7	Test application	http://localhost:5000 and http://localhost:5000/health
8	Inspect logs	docker logs flask-container
9	Inspect container	docker exec -it flask-container sh; pwd; ls; exit
10	Stop and verify	docker stop flask-container; docker ps; docker ps -a
11	Restart	docker start flask-container; docker ps
12	Cleanup	docker stop flask-container; docker rm flask-container; docker rmi flask-docker-app:v1

Output

The practical was successfully executed. Docker Desktop was running and Docker version 29.7.2 was verified. The Flask application was tested locally, built into the image flask-docker-app:v1, run as flask-container with host port 5000 mapped to container port 5000, tested through the browser, inspected using logs and docker exec, restarted, and finally cleaned up.

The successful container output showed 0.0.0.0:5000->5000/tcp, confirming the port mapping. The application and /health endpoint responded successfully at localhost:5000 and localhost:5000/health.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> docker --version
Docker version 29.7.2, build a7dcaa6
PS C:\WINDOWS\system32> docker info
Client:
Version:      29.7.2
Context:      desktop-linux
Debug Mode:   false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
  Version:    v1.124.0
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-agent.exe
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
  Version:    v1.30.0
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
  Version:    v0.36.1-desktop.1
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
  Version:    v5.4.0
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
  Version:    0.0.47
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
  Version:    v0.4.3
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-desktop.exe
dhi: CLI for managing Docker Hardened Images (Docker Inc.)
  Version:    v0.0.7
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-dhi.exe
extension: Manages Docker extensions (Docker Inc.)
  Version:    v0.2.31
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
  Version:    v1.4.0
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
  Version:    v0.43.3
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
  Version:    v1.2.6
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-model.exe
offload: Docker Offload (Docker Inc.)
  Version:    v0.6.13
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-offload.exe
pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
  Version:    v0.2.1
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-pass.exe
sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
  Version:    v0.13.0
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-sandbox.exe
scout: Docker Scout (Docker Inc.)
  Version:    v1.24.0
  Path:       C:\Users\Admin\.docker\cli-plugins\docker-scout.exe

```

```

PS C:\WINDOWS\system32> docker run hello-world

```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

```

PS C:\WINDOWS\system32> |

```

FIGURE 0: Docker Verification

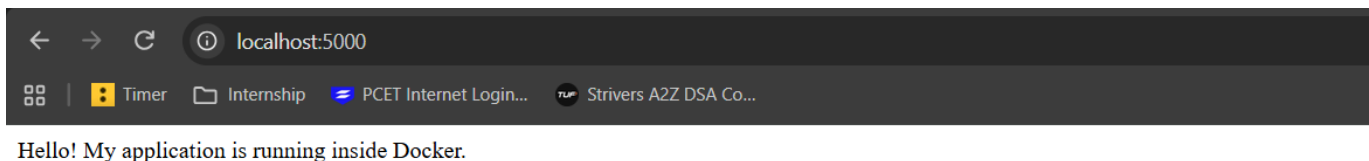


FIGURE 1: Local Flask Application

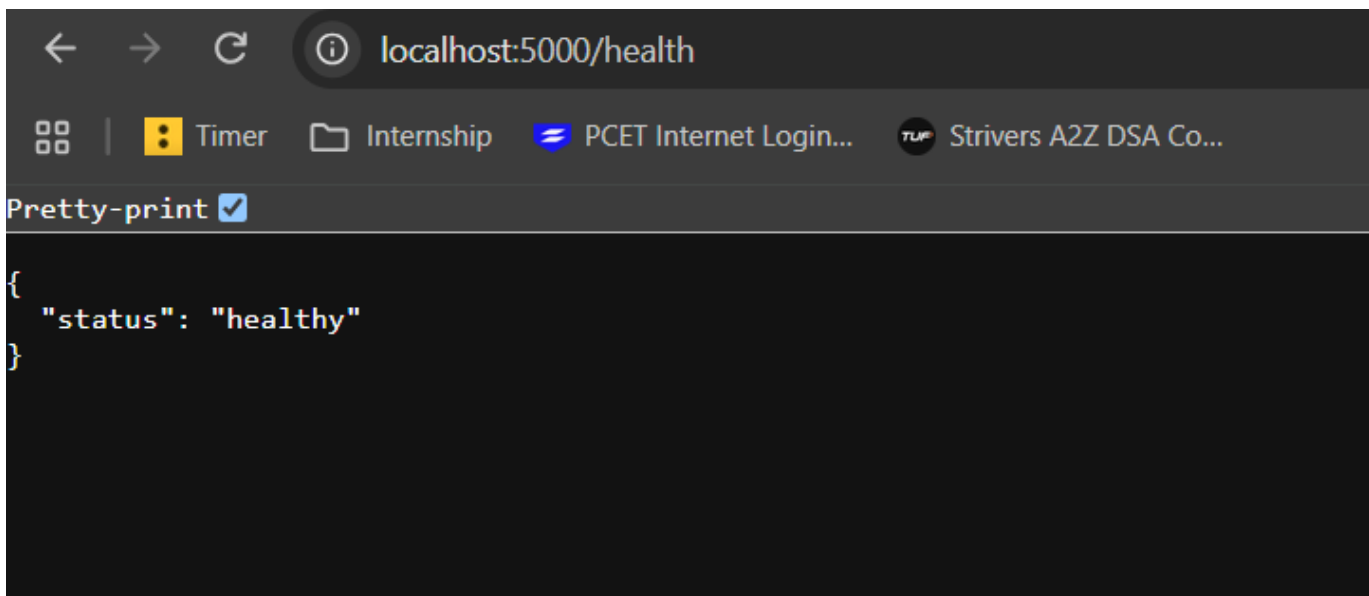


FIGURE 2: Local Flask Health Check — /health

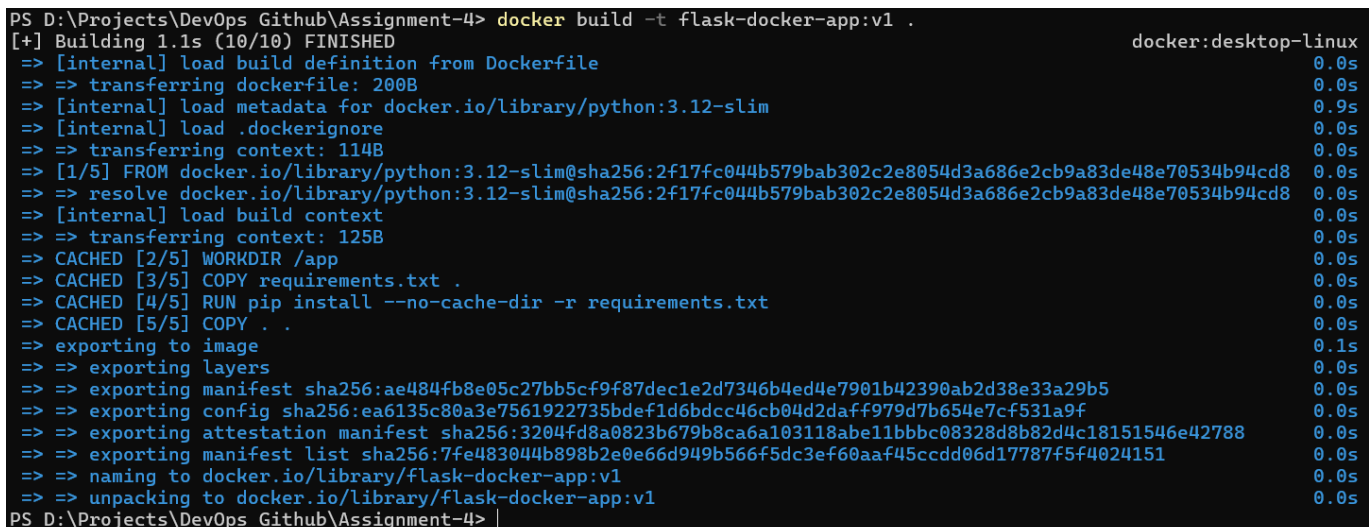


FIGURE 3: Successful Docker Image Build

```
PS D:\Projects\DevOps Github\Assignment-4> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
campuscare-backend:latest	2bfabdbe3641	217MB	51.6MB	
campuscare-frontend:latest	77f3fb1c4ae7	95.9MB	27.1MB	
flask-docker-app:v1	7fe483044b89	198MB	48.2MB	
hello-world:latest	7f4da0fc94bc	25.9kB	9.49kB	U
mongo:7	b6421fd6d1c5	1.18GB	299MB	
postgres:alpine	d3e1620b530c	433MB	121MB	U

```
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 4: Docker Image Verification

```
PS D:\Projects\DevOps Github\Assignment-4> docker run -d -p 5000:5000 --name flask-container flask-docker-app:v1
1bf109cce3640107de7ca7ec0d753db22b512973b7bd6ddb4cb51e0af9d0e59
PS D:\Projects\DevOps Github\Assignment-4> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
1bf109cce364	flask-docker-app:v1	"python app.py"	6 seconds ago	Up 5 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp

```
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 5: Running Docker Container

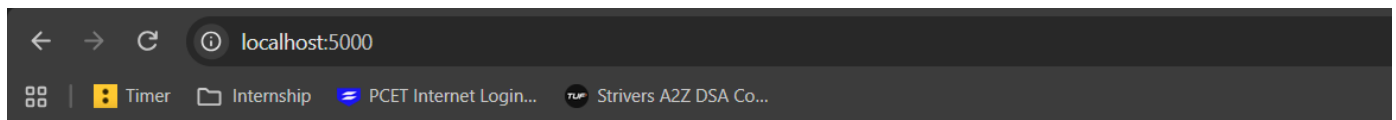


FIGURE 6: Dockerized Application

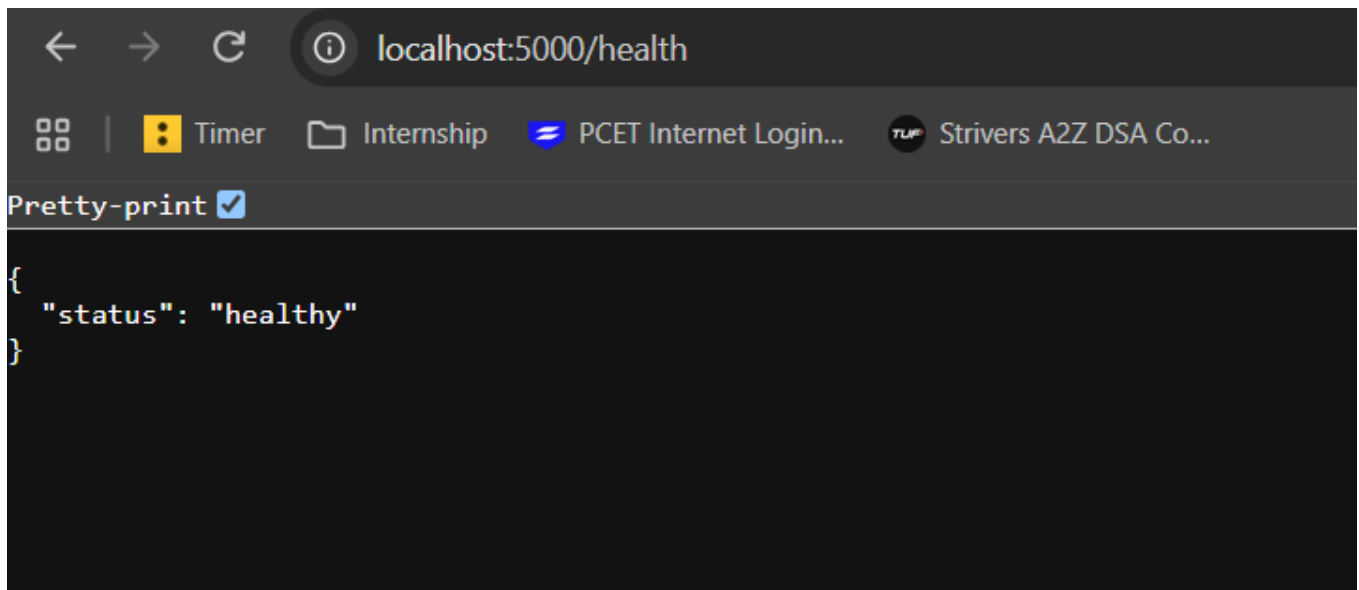


FIGURE 7: Dockerized Health Check

```
PS D:\Projects\DevOps Github\Assignment-4> docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [25/Sep/2026 19:59:23] "GET / HTTP/1.1" 200 -
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 8: Container Logs

```
PS D:\Projects\DevOps Github\Assignment-4> docker exec -it flask-container sh
# ls
Dockerfile  app.py  requirements.txt
# exit

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug flask-container
Learn more at https://docs.docker.com/go/debug-cli/
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 9: Container Shell Inspection

```
PS D:\Projects\DevOps Github\Assignment-4> docker stop flask-container
flask-container
PS D:\Projects\DevOps Github\Assignment-4> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
PS D:\Projects\DevOps Github\Assignment-4> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
1bf109cce364   flask-docker-app:v1   "python app.py"         4 minutes ago    Exited (137) 2 seconds ago    flask-container
5a853d03835c   hello-world           "/hello"               10 minutes ago    Exited (0) 10 minutes ago    friendly_hertz
a22ec1fa4c80   postgres:alpine       "docker-entrypoint.s..." 4 weeks ago    Exited (0) 4 weeks ago    campus-postgres
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 10: Container Stop and Status

```
PS D:\Projects\DevOps Github\Assignment-4> docker start flask-container
flask-container
PS D:\Projects\DevOps Github\Assignment-4> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
1bf109cce364   flask-docker-app:v1   "python app.py"         5 minutes ago    Up 1 second    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    flask-container
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 11: Container Restart

```
PS D:\Projects\DevOps Github\Assignment-4> docker stop flask-container
flask-container
PS D:\Projects\DevOps Github\Assignment-4> docker rm flask-container
flask-container
PS D:\Projects\DevOps Github\Assignment-4> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
5a853d03835c   hello-world           "/hello"               12 minutes ago    Exited (0) 12 minutes ago    friendly_hertz
a22ec1fa4c80   postgres:alpine       "docker-entrypoint.s..." 4 weeks ago    Exited (0) 4 weeks ago    campus-postgres
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 12: Container Removal

```
PS D:\Projects\DevOps Github\Assignment-4> docker rmi flask-docker-app:v1
Untagged: flask-docker-app:v1
Deleted: sha256:7fe483044b898b2e0e66d949b566f5dc3ef60aaf45ccdd06d17787f5f4024151
PS D:\Projects\DevOps Github\Assignment-4> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
campuscare-backend:latest	2bfabdbe3641	217MB	51.6MB	
campuscare-frontend:latest	77f3fb1c4ae7	95.9MB	27.1MB	
hello-world:latest	7f4da0fc94bc	25.9kB	9.49kB	U
mongo:7	b6421fd6d1c5	1.18GB	299MB	
postgres:alpine	d3e1620b530c	433MB	121MB	U

```
PS D:\Projects\DevOps Github\Assignment-4> |
```

FIGURE 13: Docker Image Cleanup

Conclusion:

In this assignment, I learned how to containerize a Python Flask application using Docker. I verified the application before Docker migration, created and used a Dockerfile and .dockerignore, built a Docker image, ran the application in a container, mapped port 5000, tested the application and health endpoint, viewed container logs, inspected the container using docker exec, demonstrated stop/start operations, and performed final cleanup. The practical demonstrated how Docker packages application dependencies into a consistent and portable deployment environment.

References:

1. Docker Documentation — <https://docs.docker.com/>
2. Dockerfile Reference — <https://docs.docker.com/reference/dockerfile/>
3. Docker CLI Reference — <https://docs.docker.com/reference/cli/docker/>
4. Python Flask Documentation — <https://flask.palletsprojects.com/>

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-4>

Hosted / Output Link: NA – Docker application executed locally